

Alcohol Label Verification

README — setup, structure, and how to run it locally

Live demo: <https://alcohol-label-verification-fawn.vercel.app>

Repository: github.com/aaryannaz/alcohol-label-verification

Summary

A web app that automates what a TTB reviewer does by hand on a COLA (Certificate of Label Approval) application: it reads the regulated fields off the label artwork, compares them to the approved application, and flags anything that does not match. The reviewer uploads the label image(s); Google Gemini Vision reads them into structured fields and pre-fills an editable set of COLA application fields; the reviewer corrects those to the approved application; a pure, rule-based engine then compares them field-by-field and runs the TTB compliance checks (required fields per beverage type, the exact government-warning text, additive disclosures, and more). Typical extraction is about 2 seconds; a batch mode processes many labels in parallel, each landing on a Pass / Needs-attention verdict.

Backend: FastAPI (Python 3.12). AI: Google Gemini Vision (gemini-2.5-flash). Frontend: vanilla HTML/CSS/JS served by the same app. Deployed on Vercel.

File structure

alcohol-label-verification/	
README.md	APPROACH.md
requirements.txt	LICENSE
pyproject.toml	docs (this PDF mirrors README.md)
vercel.json	runtime deps (installed by Vercel)
.vercelignore	packaging + ruff (lint) config
.python-version	deploy / routing config
.env.example	Python 3.12; copy to .env
.github/workflows/ci.yml	CI: ruff + tests on every push
app/	
main.py	FastAPI app: routes, exception handlers, middleware
fields.py	canonical regulated-field list (single source of truth)
schemas.py	Pydantic request models + product / origin enums
clients.py	Gemini client + model / timeout config
extraction.py	image -> JSON field extraction (Gemini, category-aware)
prompts.py	the extraction prompts (most domain rules live here)
validation.py	rule-based field comparison + TTB compliance logic
uploads.py	upload validation (extension / content-type / signature)
security.py	rate limiting, optional auth, security headers
observability.py	structured logging + per-request correlation IDs
errors.py	one JSON error envelope for all failures
static/	browser UI (HTML/CSS/JS) served by FastAPI
evals/	extraction-accuracy harness (render, score, cases)
tests/	unit + API tests (test_validation.py, test_api.py)
scripts/	gemini_smoke_test.py (Gemini connectivity check)

Run it locally (step by step)

1. Prerequisites. Python 3.12 and a Google Gemini API key (free from Google AI Studio: aistudio.google.com/apikey).

2. Get the code.

```
git clone https://github.com/aaryannaz/alcohol-label-verification.git
cd alcohol-label-verification
```

3. Set your API key. Copy the example env file and add your key.

```
cp .env.example .env
# then edit .env and set GEMINI_API_KEY=your-key
```

4. Create a virtualenv and install dependencies.

```
python -m venv venv
venv/bin/python -m pip install -r requirements.txt
```

5. Run the server (from the repository root).

```
venv/bin/python -m uvicorn app.main:app --reload
```

6. Open the app. UI at <http://localhost:8000/> · API docs at <http://localhost:8000/docs>

7. (Optional) Run the tests and the accuracy eval.

```
venv/bin/python -m unittest discover tests          # unit + API tests
venv/bin/python -m evals.run_eval                  # extraction accuracy + latency
```